RADIT: An RFID-based Student Wallet System

Real-time Research Project/ Societal Related Project /Field-Based Research Project (CS456PC)

Submitted

In partial fulfilment of the requirements for completion of

Bachelor of Technology

in

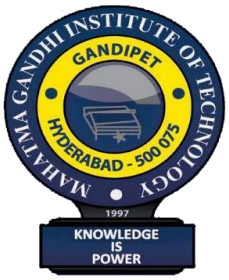
Computer Science Engineering

by

Kandregula Abhiram (22261A0586)
and
Konreddy Yaswanth Venkata Bhramha Reddy (22261A0593)

Under the guidance of

Dr B Poornima
Assistant Professor
and
Mrs. N. Musrat Sultana
Assistant Professor



Department of Computer Science and Engineering,
MAHATMA GANDHI INSTITUTE OF TECHNOLOGY,
GANDIPET, HYDERABAD-500 075, INDIA.

2025-2026

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY

(Affiliated to Jawaharlal Nehru Technological University Hyderabad)
Gandipet, Hyderabad-500 075, Telangana (India)

CERTIFICATE

This is to certify that the Real-time Research Project/ Societal Related Project /Field-Based Research Project (CS456PC) entitled "**RADIT: An RFID based Student Wallet System",** is being submitted by **Mr. Kandregula Abhiram** bearing **Roll No: 22261A0586** and **Mr. Konreddy Yaswanth Venkata Bhramha Reddy** bearing **Roll No: 22261A0593** in partial fulfillment for completion of **Bachelor of Technology VI Semester** in **Computer Science and Engineering** to **Mahatma Gandhi Institute of Technology** is a record of Bonafide work carried out by him under our guidance and supervision. The results embodied in this project have not been submitted to any other University or Institute for the award of any degree or diploma.

Project Guide
Mrs. P Poornima
Asst. Professor, Dept. of CSE

Head of the Department
Dr. C.R.K Reddy
Professor, Dept. of CSE

Project Guide
Mrs. N. Musrat Sultana
Asst. Professor, Dept. of CSE

DECLARATION

This is to certify that the work reported in Real-time Research Project/ Societal Related Project /Field-Based Research Project (CS456PC) titled **"RADIT: An RFID based Student Wallet System"** is a record of work done by us in the Department of Computer Science and Engineering, Mahatma Gandhi Institute of Technology, Hyderabad. No part of the work is copied from books/journals/internet and wherever the portion is taken, the same has been duly referred to in the text. The report is based on the work done entirely by us and not copied from any other source.

KANDREGULA ABHIRAM
(22261A0586)

KONREDDY YASWANTH VENKATA BHRAMHA REDDY
(22261A0593)

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people who made it possible because success is the abstract of hard work and perseverance, but steadfast of all is encouraging guidance. So, we acknowledge all those whose guidance and encouragement served as a beacon light and crowned my efforts with success.

.

We would like to express our sincere thanks to, **Prof G. Chandra Mohan Reddy, Principal MGIT**, for providing the working facilities in college.

We wish to express our sincere thanks and gratitude to **Dr. C R K Reddy, Professor and HoD**, **Department of CSE, MGIT**, for all the timely support and valuable suggestions during the period of project.

We are extremely thankful to **Dr B. Poornima, Assistant Professor, Department of CSE, MGIT and Mrs K Vedavathi, Assistant Professor, Department of CSE, MGIT,** Real-time Research Project/ Societal Related Project /Field-Based Research Project Coordinators for their encouragement and support throughout the project.

We are also extremely thankful and indebted to our mentors, **Dr B. Poornima, Assistant Professor, Department of CSE, MGIT and Mrs N. Musrat Sultana, Assistant Professor, Department of CSE, MGIT**, for their constant guidance, encouragement, and moral support throughout the project.

Finally, we would also like to thank all the faculty and staff of the CSE Department who helped us directly or indirectly in completing this project.

KANDREGULA ABHIRAM
(22261A0586)

KONREDDY YASWANTH VENKATA BHRAMHA REDDY

(22261A0593)

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

CPU	: Central Processing Unit
GB	: Gigabytes
Wi-Fi	: Wireless Fidelity
NoSQL	: Not Only SQL
SQL	: Structured Query Language
VS Code	: Visual Studio Code
RFID	: Radio-Frequency Identification
IoT	: Internet of Things
GUI	: Graphical User Interface
PIN	: Personal Identification Number
HTTPS	: Hypertext Transfer Protocol Secure
GPS	: Global Positioning System
GSM	: Global System for Mobile Communications
SMS	: Short Message Service
ER	: Entity Relationship
CRUD	: Create, Read, Update, Delete

ABSTRACT

The RADIT system represents a pioneering approach to campus management that transforms chaotic administrative processes into a streamlined, technology-driven ecosystem. Leveraging RFID technology coupled with biometric verification, this comprehensive platform creates a secure, cashless environment for students across four essential campus modules. The digital wallet serves as the central hub, enabling seamless financial transactions for canteen purchases, stationery items, and miscellaneous college expenses, eliminating physical cash handling while providing usage pattern-based wallet top-up suggestions. The library management system incorporates book recommendations based on student borrowing history and academic interests. The bus tracking module delivers real-time updates to parents regarding student transportation as IoT devices such as GPS, GSM, and RFID, enable real-time location tracking, route optimization, and attendance tracking for school buses [2], while the classroom attendance system employs facial recognition to ensure accurate student verification. RADIT's cloud-based architecture ensures scalability, reliability, and real-time data synchronization across all modules, with recommendation systems providing personalized wallet recharge recommendations based on historical spending patterns. By centralizing these services on a secure cloud platform, RADIT enhances operational efficiency, strengthens security protocols, reduces manual paperwork, and generates valuable data analytics for institutional decision-making, creating a more organized, responsive, and student-centered smart campus experience.

Keywords: RFID technology, cloud computing, usage pattern-based recommendations, cashless campus, biometric verification, smart campus

1. INTRODUCTION

RADIT is a comprehensive, modular campus management system designed to streamline and unify essential student services within educational institutions. By leveraging IoT based technologies like RFID technology and biometric verification [7, 14], and cloud-based data storage, RADIT provides a secure, cashless, and highly efficient environment for students, administrators, and service providers. The system integrates four core modules—digital wallet, library management, bus tracking, and classroom attendance—into a single, user-friendly platform. Through usage pattern-based recommendation systems, RADIT enhances the student experience by offering personalized suggestions for wallet recharges and book borrowing, while real-time data synchronization ensures up-to-date information across all modules. The result is a connected, organized, and responsive campus ecosystem that eliminates manual processes and data silos, ultimately improving operational efficiency and campus safety.

1.1 PROBLEM DEFINITION

Traditional campus management in educational institutions is often fragmented, relying on separate systems or manual processes for handling student finances, library operations, transportation, and attendance. This fragmentation leads to inefficiencies, increased administrative workload, and a poor user experience for students and staff. Students must manage physical cash for daily transactions, increasing security risks and complicating expense tracking. Library operations are hampered by isolated systems, making book lending and returns cumbersome. Bus tracking lacks real-time updates, causing uncertainty for parents and students [9]. Attendance systems are vulnerable to proxy attendance and require manual verification [8]. The absence of a unified, digital platform results in data silos, redundant efforts, and limited insight into campus operations, highlighting the need for an integrated solution that centralizes and automates these critical services.

1.2 EXISTING SYSTEMS

Several solutions exist for campus management, such as Blackboard, Canvas, Koha (library management), and SafeBus (transportation tracking). However, these platforms typically address only specific aspects of campus life and often lack integration between modules. Many are costly, complex, or not tailored for smaller institutions. Furthermore, most existing systems do not offer usage pattern-based recommendation features for wallet recharges or book borrowing, nor do they provide seamless integration of RFID and biometric verification for enhanced security. The lack of a unified, affordable, and user-friendly platform that brings together all essential campus services remains a significant gap in the current landscape.

1.3 PROPOSED SYSTEM

RADIT addresses these challenges by offering a unified, cloud-based campus management system [11, 12] that integrates digital wallet, library, bus tracking, and attendance modules. Built using Python (with Tkinter for the GUI) and Firebase Firestore for the backend, RADIT is

designed for ease of use, scalability, and reliability. The digital wallet enables cashless transactions for canteen, stationery, and miscellaneous expenses, with personalized recharge suggestions based on spending patterns [17]. The library module streamlines book lending and returns [3, 6], providing recommendations based on borrowing history and interests. The bus tracking module offers real-time updates to parents and students, while the attendance module uses RFID and facial recognition for secure, accurate verification. All modules are interconnected, ensuring real-time data synchronization and centralized management. RADIT's modular architecture allows for easy customization and expansion, making it suitable for institutions of varying sizes and needs.

1.4 REQUIREMENTS SPECIFICATION

Requirement Specifications describe the arti-craft of Software Requirements and Hardware Requirements used in this project.

Software Requirements

1. **Operating System:** Windows 10 or later (64-bit); Linux (Ubuntu 18.04+ or Raspberry Pi OS) is also possible
2. **Database:** Firebase Cloud Firestore
3. **Programming Language:** Python 3.8 or later
4. **Backend:** Python (with Flask optional for API [13,18], but not required in your current implementation)
5. **Frontend:** Tkinter/ttk (Python GUI); no React.js frontend in your current project
6. **Libraries/Frameworks:**
   a. firebase-admin (for Firebase integration)
   b. Pillow (image processing)
   c. OpenCV (for facial recognition)
   d. datetime, re, threading, smtplib (for various utilities)
7. **Communication Protocols:** HTTPS (for secure Firebase communication)
8. **Security:** RFID-based authentication, facial recognition, and PIN/password for admin access
9. **AI Integration:** No AI/ML libraries (like TensorFlow) are used; recommendations are based on user usage patterns, not AI

Hardware Requirements

1. **Processor:** Standard desktop/laptop CPU (e.g., Intel Core i5 or equivalent, 4–8 cores recommended)
2. **Memory:** 4GB RAM (minimum), 8GB recommended for smooth operation
3. **Storage:** 10GB free disk space (for application, dependencies, and local data caching)
4. **RFID Module:** 13.56 MHz or 125 KHz RFID reader (USB or serial interface, compatible with desktop/laptop)
5. **RFID Tags:** Passive RFID cards or key fobs
6. **Display:** Standard monitor (minimum 1366x768 resolution); touchscreen optional
7. **Connectivity:** Wi-Fi or Ethernet (for internet access to Firebase)
8. **Power:** Standard AC power supply (for desktop/laptop and peripherals)
9. **Webcam:** Required for facial recognition (minimum 720p resolution)

2. LITERATURE SURVEY

1. **"Development of a Lecture Attendance Monitoring System with Multi-Level Authentication"** *by Onwubiko, Emmauel et al.*
   This research presents an advanced attendance monitoring system employing multi-level authentication techniques including biometric fingerprint authentication and OTP mechanisms. The system addresses proxy attendance issues through multiple security layers, ensuring reliable and secure attendance tracking. The study demonstrates significant improvements in accuracy and reduction of human intervention compared to traditional methods. For our project, this research emphasizes the importance of multi-factor authentication and provides a framework for implementing robust security measures in attendance systems.

2. **“IoT-Based School Bus and Student Monitoring System Using RFID and GSRM Technologies”** *by Ranjan, Rashmi et al***.**
   This study introduces an IoT-based system that enhances school bus safety by integrating RFID and GPS tracking. RFID records students’ presence inside or outside the bus, while GPS enables real-time vehicle tracking. The system also features an emergency SOS button that sends alerts via GSM to predefined contacts. The research emphasizes automation, real-time monitoring, and parental notifications. For our project, this highlights the potential of RFID and IoT for tracking student bus usage, ensuring safety, and providing real-time updates to stakeholders.

3. **"Enhancing Library Management through RFID and IoT Integration in Nigeria: Benefits, Challenges, and Future Prospects"** *by Lucky Oji Akpojotor & Esoswo Francisca Ogbomo*
   This comprehensive study explores the transformative impact of integrating RFID and IoT technologies in Nigerian library management systems. The research demonstrates how RFID technology modernizes inventory tracking and security while IoT integration enables smart shelving, real-time analytics, and personalized user experiences. Case studies from Nigerian universities show improved operational efficiency and user satisfaction. For our project, this research provides valuable insights into the practical benefits and implementation challenges of RFID-IoT integration in educational environments, particularly highlighting the importance of addressing technical issues, privacy concerns, and cost considerations.

4. **“Biometric and RFID Passive Tag-Based Student Identification System for Secure Attendance Management”** *by N. R et al.*
   This study presents an advanced attendance management system integrating biometric authentication with RFID tags for secure and accurate student tracking. By combining fingerprint recognition with RFID-based ID cards, the system prevents proxy attendance and manual entry errors. It also includes real-time attendance updates via a GSM-based SMS notification system for guardians and a web application for monitoring student attendance and location. For our project, this highlights the effectiveness of multi-factor

authentication and automated notifications in enhancing security and reliability in student attendance tracking.

5. "**An RFID-Based Smart School Attendance and Monitoring System"** *by Farag, W. A.*
   This study proposes an RFID-based automated attendance system (RFID-AS) to enhance student tracking and evaluation. By using passive RFID technology, the system ensures cost-effectiveness while improving security and grading accuracy. The system integrates RFID readers, SQL Server for data management, and a Visual Studio-based GUI, allowing parents and faculty to monitor student records. Attendance is recorded as students pass through RFID-enabled classroom doors. For our project, this highlights the practicality of RFID for seamless attendance tracking and real-time student monitoring in educational institutions.

6. **"RFID-Based Library Management System used in Library"** *by Srinivasan, Rajasekaran et al.*
   This study presents an RFID-based library management system that enhances efficiency in book transactions, tracking, and security. By integrating RFID readers and tags, the system automates borrowing and returning, reducing manual intervention. Key benefits include preventing book losses, improving stock verification, and enabling real-time updates via GSM. For our project, this research highlights how RFID can streamline library management, ensuring accurate book tracking and automated fine management, making it a relevant model for implementing RFID-based book lending and return systems.

7. **"IoT & Cloud-based Smart Attendance Management System using RFID"** *by Samaddar, Rajarshi et al.*
   This research proposes an innovative attendance management architecture utilizing IoT, AWS cloud services, and RFID technology with Arduino Uno boards. The system automates attendance processes through hardware components including RFID modules and software built using Python Django hosted on AWS cloud. The solution provides real-time attendance tracking accessible via web and mobile applications, demonstrating superior accuracy and efficiency compared to traditional systems. For our project, this study emphasizes the effectiveness of cloud-based solutions and the integration of multiple technologies for creating robust, scalable attendance management systems.

8. **"AI System for Avoid Proxy Students"** *by Yogayya Swami et al.*
   This research presents an advanced AI-driven solution to combat proxy attendance in educational institutions using facial recognition, deep learning, and real-time monitoring. The system integrates computer vision and biometric verification to authenticate student presence accurately, preventing fraudulent attendance practices. By combining AI-driven face recognition with live detection mechanisms, the system ensures legitimate student verification. For our project, this research highlights the critical importance of preventing proxy attendance and demonstrates how AI technologies can enhance the security and integrity of attendance management systems.
9. **"Smart Bus Tracking System for Students using RFID"** *by Komal Sukraj Dambre et al.*
   This study describes a comprehensive school bus tracking system using GPS, RFID, and

GSM technologies to ensure student safety and provide parents with real-time information. The system tracks student boarding/deboarding status and provides bus location updates through SMS notifications, addressing parental concerns about student safety during transportation. For our project, this research demonstrates the practical application of RFID technology in educational safety systems and highlights the importance of real-time communication with stakeholders for enhanced security and peace of mind.

10. **"Design and Implementation of Student Information System"** *by Ike, Ifeanyi*
    This research focuses on developing efficient and effective student information management systems for educational institutions. The study emphasizes the importance of systematic data organization and management for improving administrative processes in schools and universities. For our project, this work provides foundational insights into database design and management principles essential for creating comprehensive student management systems that integrate with attendance tracking solutions.

11. **"Cloud Computing In Higher Education Institutions: Pros and Cons"** *by Helaimia, Rafika*
    This study examines the adoption of cloud computing technologies in higher education institutions, analyzing the benefits and challenges associated with cloud-based solutions. The research explores various cloud service models and deployment strategies in educational contexts. The study identifies key benefits including cost-effectiveness, scalability, and accessibility, while also highlighting challenges related to privacy, security, and infrastructure requirements. For our project, this research provides valuable insights into cloud computing implementation considerations for educational management systems.

12. **"Cloud-based Attendance Management System with Integrated RFID Data Acquisition and Real-Time Google Sheets Synchronization"** *by Aditi Akundi et al.*
    This study presents an innovative RFID-based attendance system using NodeMCU ESP8266 microcontroller and Google Sheets integration. The system automates attendance procedures through real-time data synchronization with cloud-based spreadsheets, offering a cost-effective solution using affordable hardware components. The research demonstrates successful integration of RFID technology with cloud services for efficient data management and accessibility. For our project, this work showcases the practical benefits of cloud integration and real-time data synchronization in attendance management systems.

13. **"Implementing a Flask-based Chatbot for College Enquiries using Spacy and TensorFlow"** *by P. Naresh et al.*
    This study focuses on developing intelligent chatbot systems using NLP libraries Spacy and TensorFlow within Flask-based applications. The research demonstrates the process of training deep learning models for conversational AI, including data preprocessing, tokenization, and feature extraction. For our project, this work provides insights into developing intelligent user interfaces and automated query handling systems that could enhance user experience in attendance management applications.

14. **“The Role of Internet of Things in Smart Education”** *by Valentina Terzieva, Svetozar*

Ilchev, Katia Todorova.

This study explores how IoT can transform education by creating smart learning environments. It discusses IoT's role in optimizing educational processes through sensor-based data collection and intelligent communication protocols. The research presents two IoT prototypes designed to enhance smart schools by automating data gathering and information dissemination. For our project, this highlights the integration of IoT for efficient student tracking, automated attendance, and real-time updates in an educational setting.

15. **"E-Wallet: A Study on Cashless Transactions Among University Students"** *by Chelvarayan A, Yeo SF, Hui Yi H, Hashim H.*

    This study explores university students' adoption of e-wallets using the Technology Acceptance Model (TAM). It highlights key influencing factors such as perceived usefulness, ease of use, risk, and trust. The research, based on a survey of 140 students from a Malaysian private institution, suggests that e-wallets provide a more convenient alternative to cash and traditional payment methods. Findings can help institutions, businesses, and policymakers understand students' financial behaviour and improve digital payment adoption. For our project, this study reinforces the importance of user trust, security, and convenience in implementing a student-focused digital wallet system.

16. **"RAFI: Robust Authentication Framework for IoT-Based RFID Infrastructure"** *by Kumar, V. et al.*

    This study addresses security concerns in IoT-based RFID systems by proposing a robust authentication framework. It highlights vulnerabilities in RFID communication, such as data privacy risks and security threats, and presents a model that ensures secure communication using formal security analysis. The research emphasizes the need for authentication mechanisms in RFID infrastructure to prevent unauthorized access. For our project, this underscores the importance of implementing secure authentication and encryption methods in RFID-based student tracking and transaction systems.

17. **"A College Student Behaviour Analysis and Management Method Based on Machine Learning Technology"** *by Shen, Xiaoying & Yuan, Chao.*

    This study explores how machine learning, specifically an adaptive K-means algorithm, can analyse student behaviour in digital campuses. By clustering data related to study patterns, lifestyle, and financial habits, the system identifies issues like excessive online time, low book borrowing rates, and financial constraints. The insights help institutions improve student management, safety, and academic performance. For our project, this highlights the potential of using data-driven decision-making for personalized student support, optimizing resource allocation, and enhancing financial monitoring in an RFID-based campus system.

18. **"Efficient Way Of Web Development Using Python And Flask"** *by Aslam, Fankar et al.*

    This study focuses on the technological aspects of web portal development using Python

and Flask frameworks, emphasizing the importance of appearance and user experience in web applications. The research demonstrates how proper technology selection contributes to creating well-structured, visually appealing web portals that attract more users. For our project, this work provides essential insights into web development best practices and the effectiveness of Python-Flask combination for creating user-friendly interfaces for attendance and management systems.

19. **"An Effective Framework for Managing University Data using a Cloud based Environment"** *by Kashish Ara Shakil et al.*
    This research proposes an effective framework for managing university data using cloud-based environments, addressing the challenges of data management in large educational institutions. The study presents a cloud data management simulator demonstrating the applicability of cloud computing in educational sectors. The framework includes support for modeling cloud infrastructure, user-friendly interfaces, and virtualized access to educational data. For our project, this research provides a comprehensive understanding of cloud-based data management strategies essential for scalable educational systems.

20. **"Biometric Authentication: A Review"** *by Bhattacharyya, Debnath et al.*
    This comprehensive review paper examines various biometric authentication techniques and their applications in security systems. The study analyzes physiological parameters used for human identification and compares different biometric methods, discussing their advantages and disadvantages. The research emphasizes the role of biometrics in ensuring secure access control and preventing unauthorized system access. For our project, this review provides essential background on biometric technologies and their effectiveness in creating secure authentication systems for educational environments.

2.1 LITERATURE SURVEY TABLE

Table 2.1: Literature Survey Table

S. No	Author(s)	Title	Year	Merits	Demerits
1	Onwubiko, Emmauel et al.	Development of a Lecture Attendance Monitoring System with Multi-Level Authentication	2025	Implements robust multi-factor authentication with biometric and OTP verification for enhanced security.	Complex system requiring multiple authentication steps which may slow down attendance marking process.
2	Ranjan, Rashmi et al.	IoT-Based School Bus and Student Monitoring System Using RFID and GSM Technologies	2024	Provides real-time tracking of school buses and student presence via RFID and GPS; includes an emergency SOS alert system.	The study is more focused on school environments, and additional applications in higher education institutions can be examined.
3	Lucky Oji Akpojotor & Esoswo Francisca Ogbomo	Enhancing Library Management through RFID and IoT Integration in Nigeria: Benefits, Challenges, and Future Prospects	2024	Demonstrates practical benefits of RFID-IoT integration in Nigerian libraries with improved efficiency and user satisfaction.	Implementation challenges include technical issues, privacy concerns, and high costs requiring careful consideration.
4	N. R et al.	Biometric and RFID Passive Tag-Based Student Identification System for Secure Attendance Management	2023	Combines biometrics and RFID for accurate attendance tracking, preventing fraud; includes SMS alerts for guardians.	The study incorporates biometric authentication, and its scalability in larger institutions can be further studied.
5	Farag, W. A.	An RFID-Based Smart School Attendance and Monitoring System	2023	Uses RFID for automated attendance, reducing manual errors; provides a GUI for parent and faculty monitoring.	The system is primarily designed for school settings, and additional modifications can enhance its application in broader educational contexts.
6	Srinivasan, Rajasekaran et al.	RFID-Based Library Management System used in Library	2023	Automates library transactions, enhances security, and prevents book loss; improves stock verification with RFID.	The study primarily focuses on library management, and further integration with multi-functional student systems can be explored.
7	Samaddar, Rajarshi et al.	IoT & Cloud-based Smart Attendance Management System using RFID	2023	Provides real-time attendance tracking with cloud integration using AWS and Python Django; demonstrates superior accuracy over traditional systems.	System reliability depends on internet connectivity and cloud service availability.
8	Yogayya Swami et al.	AI System for Avoid Proxy Students	2023	Utilizes advanced facial recognition and deep	Requires significant computational resources

				learning to prevent proxy attendance with high accuracy and reliability.	and may face challenges with lighting conditions and image quality.
9	Komal Sukraj Dambre et al.	Smart Bus Tracking System for Students using RFID	2023	Provides comprehensive school bus safety solution with GPS tracking, RFID-based boarding detection, and SMS notifications for parents.	System effectiveness depends on GPS signal availability and GSM network coverage.
10	Ike, Ifeanyi	Design and Implementation of Student Information System	2023	Focuses on efficient student data organization and management for improved administrative processes.	Limited integration capabilities with other educational technology systems.
11	Helaimia, Rafika	Cloud Computing In Higher Education Institutions: Pros and Cons	2023	Thorough examination of cloud adoption in education with analysis of service models and deployment strategies.	Focuses on general cloud computing without specific implementation guidelines for attendance systems.
12	Aditi Akundi et al.	Cloud-based Attendance Management System with Integrated RFID Data Acquisition and Real-Time Google Sheets Synchronization	2023	Demonstrates cost-effective solution using affordable hardware with real-time cloud synchronization capabilities.	Limited to Google Sheets integration which may not be suitable for enterprise-level implementations.
13	P. Naresh et al.	Implementing a Flask-based Chatbot for College Enquiries using Spacy and TensorFlow	2023	Provides framework for intelligent user interface development with conversational AI capabilities.	Requires significant training data and computational resources for effective NLP implementation.
14	Valentina Terzieva, Svetozar Ilchev, Katia Todorova	The Role of Internet of Things in Smart Education	2022	Explores IoT's role in smart education and presents prototypes for data collection and analysis in learning environments.	The research primarily introduces IoT applications, and further case studies can enhance practical implementation.
15	Chelvarayan A, Yeo SF, Hui Yi H, Hashim	E-Wallet: A Study on Cashless Transactions Among University Students	2022	Identifies key factors affecting e-wallet adoption among students; provides insights for institutions and businesses to enhance digital payment systems.	The study is limited to a specific group of students, and additional influencing factors can be explored.
16	Kumar, V. et al.	RAFI: Robust Authentication Framework for IoT-Based RFID Infrastructure	2022	Proposes a secure authentication framework for RFID-based IoT systems, addressing security threats and	The study focuses on security frameworks, and additional real-world implementation scenarios can further validate the approach.

				privacy concerns.	
17	Shen, Xiaoying & Yuan, Chao	A College Student Behaviour Analysis and Management Method Based on Machine Learning Technology	2021	Uses machine learning (adaptive K-means) to analyse student behaviour based on financial, academic, and lifestyle data.	The study focuses on behaviour analysis, and further exploration of predictive analytics can provide deeper insights.
18	Aslam, Fankar et al.	Efficient Way Of Web Development Using Python And Flask	2015	Emphasizes importance of technology selection for creating visually appealing and well-structured web applications.	Limited to web development aspects without comprehensive system integration details.
19	Kashish Ara Shakil et al.	An Effective Framework for Managing University Data using a Cloud based Environment	2015	Comprehensive framework for educational data management with support for modeling cloud infrastructure and virtualized access.	—
20	Bhattacharyya, Debnath et al.	Biometric Authentication: A Review	2009	Comprehensive analysis of various biometric methods with detailed comparison of advantages and disadvantages.	Theoretical review without practical implementation examples in educational contexts and outdated research.

3. METHODOLOGY

This section outlines the approach taken to design, develop, and implement the RADIT system, ensuring a robust, scalable, and user-friendly campus management solution.

3.1 TECHNOLOGIES USED

a) **Programming Language:** Python 3.8 or later Python was chosen for its readability, extensive library support, and ease of integration with both hardware (RFID, webcam) and cloud services.
b) **GUI Framework:** Tkinter/ttk Tkinter provides a lightweight, native interface for building desktop applications, ensuring cross-platform compatibility and ease of use for both students and administrators.
c) **Database:** Firebase Cloud Firestore Firestore offers a scalable, real-time, cloud-based NoSQL database, enabling secure storage and instant synchronization of student, transaction, library, attendance, and bus activity data [19].
d) **Image Processing & Face Recognition:** OpenCV, Pillow OpenCV is used for facial recognition in attendance verification, while Pillow handles image processing tasks such as resizing and format conversion.
e) **RFID Integration:** Standard USB/Serial RFID Reader The system interfaces with a 13.56 MHz or 125 KHz RFID reader for secure student identification and transaction authentication.
f) **Email Notifications:** smtplib Used for sending real-time notifications to parents regarding bus boarding and exit events.
g) Other Libraries:
    - firebase-admin for Firebase integration
    - datetime , re , threading for utility functions and background processing

3.2 DEVELOPMENT PROCESS

The development of RADIT followed an iterative and modular approach, ensuring flexibility and continuous improvement throughout the project lifecycle:

a) **Requirement Analysis**: Stakeholder needs were gathered and analyzed to define the system's core modules: digital wallet, library, bus tracking, and attendance.
b) **System Design:** The architecture was planned with a focus on modularity, allowing each service (wallet, library, bus, attendance) to function independently yet integrate seamlessly via a shared database and authentication system.
c) Module Implementation:
    - **Digital Wallet:** Developed first to establish the core RFID authentication and transaction logic.
    - **Library Management:** Integrated book lending/return, catalogue management, and usage pattern-based recommendations.
    - **Bus Tracking:** Implemented RFID-based boarding/exit logging and parent notification.
    - **Attendance:** Added facial recognition for secure, proxy-proof attendance

marking.

d) **Database Integration:** Firebase Firestore collections were structured for students, transactions, books, lendings, bus activities, and attendance, ensuring efficient data retrieval and security.
e) **Testing:** Each module underwent unit and integration testing. User acceptance testing was conducted with sample data to validate workflows and user experience.
f) **Deployment & Feedback:** The system was deployed in a controlled environment, and feedback was collected from users (students, mentors and project guides) for further refinement.

4. DESIGN OF RADIT

The **RADIT** system is designed as a **modular, role-based** platform for student authentication, attendance tracking, cashless transactions [15], library management, and bus onboarding/offboarding. It follows a **scalable and maintainable** architecture with key components including **database design, modular application structure, user interface, and role-based access control**.

4.1. SYSTEM ARCHITECTURE

Key Components:

- **RFID Scanner:** Reads student IDs for authentication.
- **Application Modules:** Attendance, Wallet, Library, and Bus tracking.
- **Firebase Database:** Stores student records, transactions, and logs.
- **Graphical User Interface (GUI):** Role-based dashboards for different users.

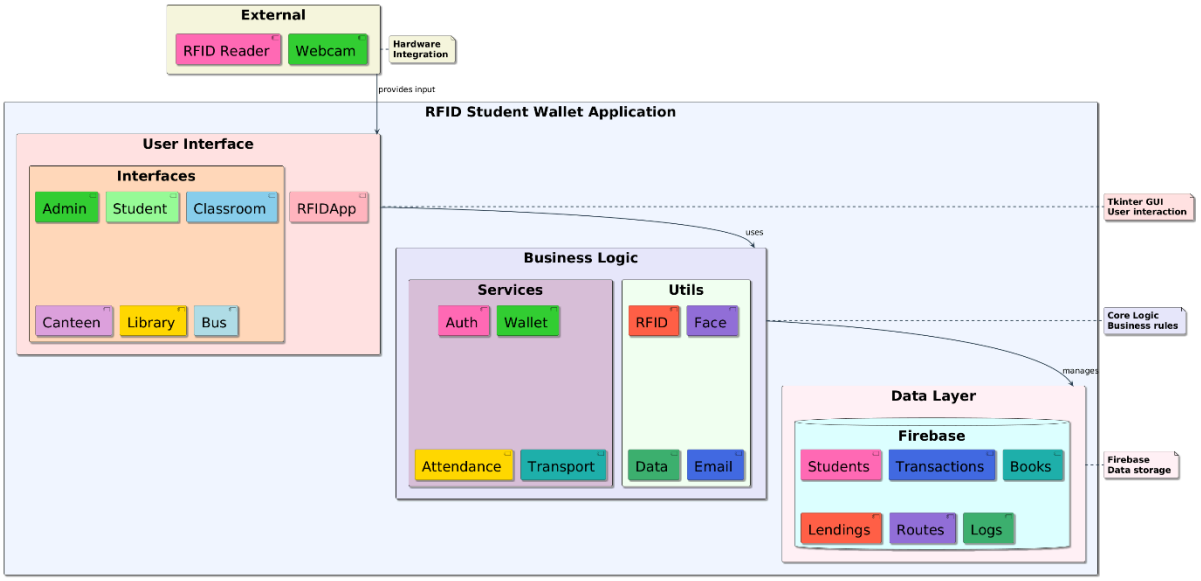


Figure 4.1: System architecture diagram

The system consists of multiple components interacting through **RFID authentication and a database-driven backend**. It is shown in Figure 4.1.

4.2. ENTITY-RELATIONSHIP AND DATABASE SCHEMA

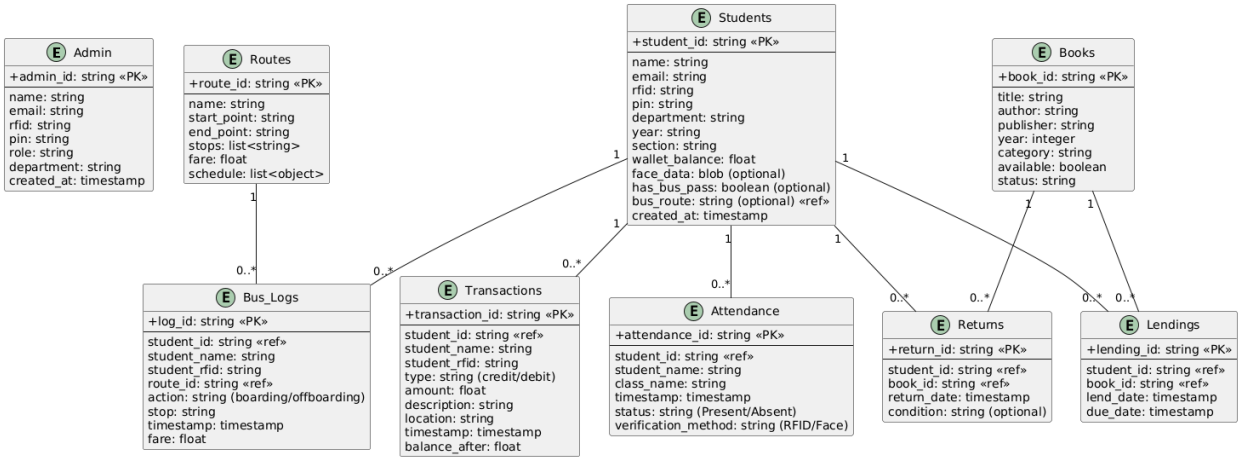
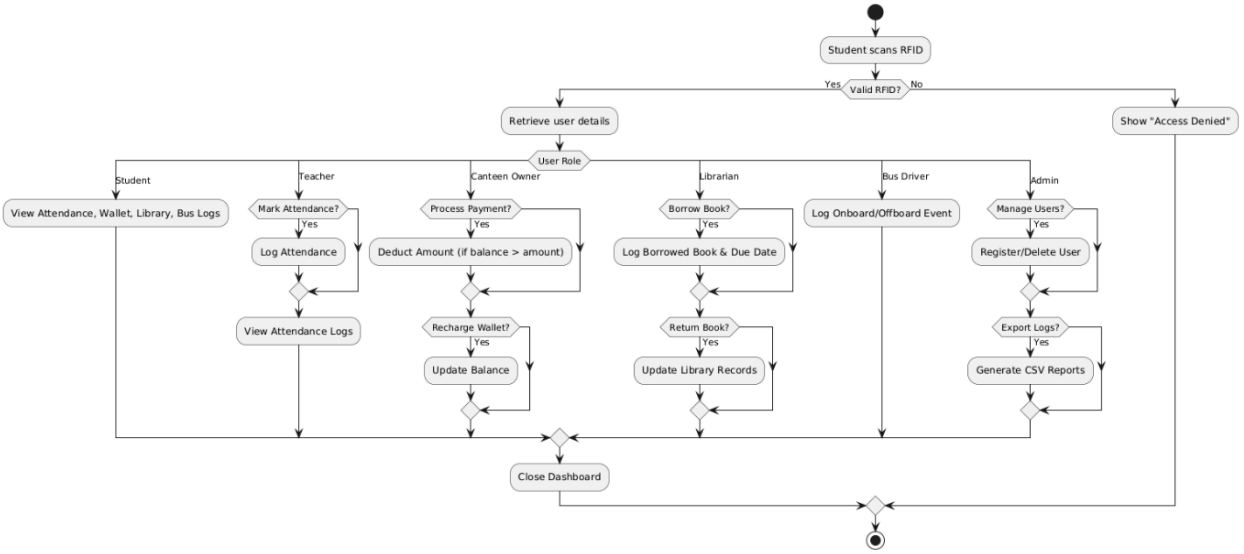


Figure 4.2: ER diagram

The Figure 4.2 is an Entity Relationship Diagram (ERD) illustrating a Firebase Firestore database schema. It defines entities such as Admin, Routes, Students, Books, Bus_Logs, Attendance, Transactions, Lendings, and Returns. The diagram shows the attributes of each entity, including data types and primary keys (PK). Relationships between entities are depicted with lines and cardinality notations (e.g., 1, 0..*). The schema model is a system involving students, books, transactions, and bus routes.

4.3. ACTIVITY DIAGRAM

Figure 4.3: Activity diagram



The **Activity Diagram** shown in Figure 4.3 illustrates the step-by-step execution of system processes such as **attendance marking, transactions, library book tracking, and bus log updates.**

4.4. MODULES

The **RFID Student Wallet System** is designed using a **modular and object-oriented** approach for flexibility and scalability. Each module represents a distinct system functionality, with well-defined **classes and relationships** to ensure smooth integration.

Key Modules & Their Corresponding Classes

- **Student (`student.py`)** → Stores student details and logs activities.
  - **Class:** `Student (rfid_uid, name, role, balance)`
- **Attendance (`attendance.py`)** → Logs RFID-based attendance.
  - **Class:** `Attendance (rfid_uid, timestamp)`
- **Bus (`bus.py`)** → Tracks student onboarding/offboarding.
  - **Class:** `Bus (rfid_uid, stop_name, route_number, action)`
- **Library (`library.py`)** → Manages book borrow/return actions.
  - **Class:** `Library (rfid_uid, book_name, due_date, action)`
- **Wallet (`wallet.py`)** → Handles cashless transactions.
  - **Class:** `Transaction (rfid_uid, amount, timestamp)`
- **Admin (`admin.py`)** → Manages users and system data.
  - **Class:** `Admin (registerUser(), deleteUser(), exportLogs())`
- **Graphical UI (`main.py`)** → Provides **role-based dashboards** for students, teachers, canteen owners, librarians, drivers, and admins.

Class Diagram Overview

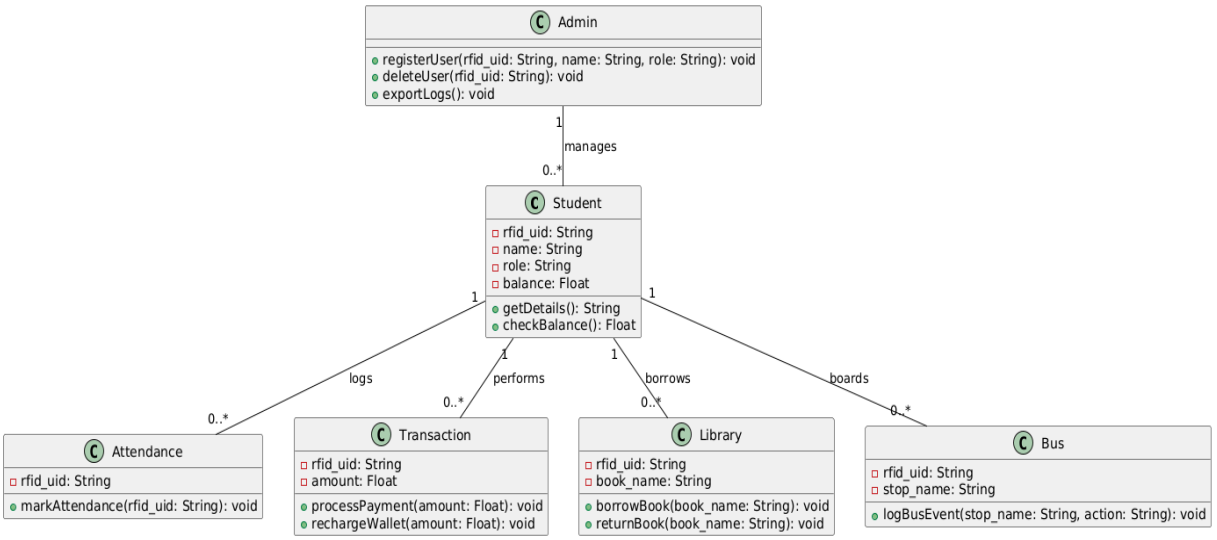


Figure 4.4: Class diagram

The **Class Diagram shown in Figure 4.4** visually represents the relationships between these classes:

- **Student** interacts with **Attendance, Wallet, Bus, and Library**.
- **Admin** manages **users and system operations**.
- **Transactions, Logs, and Events** are linked to respective modules.

4.5. USER ROLES

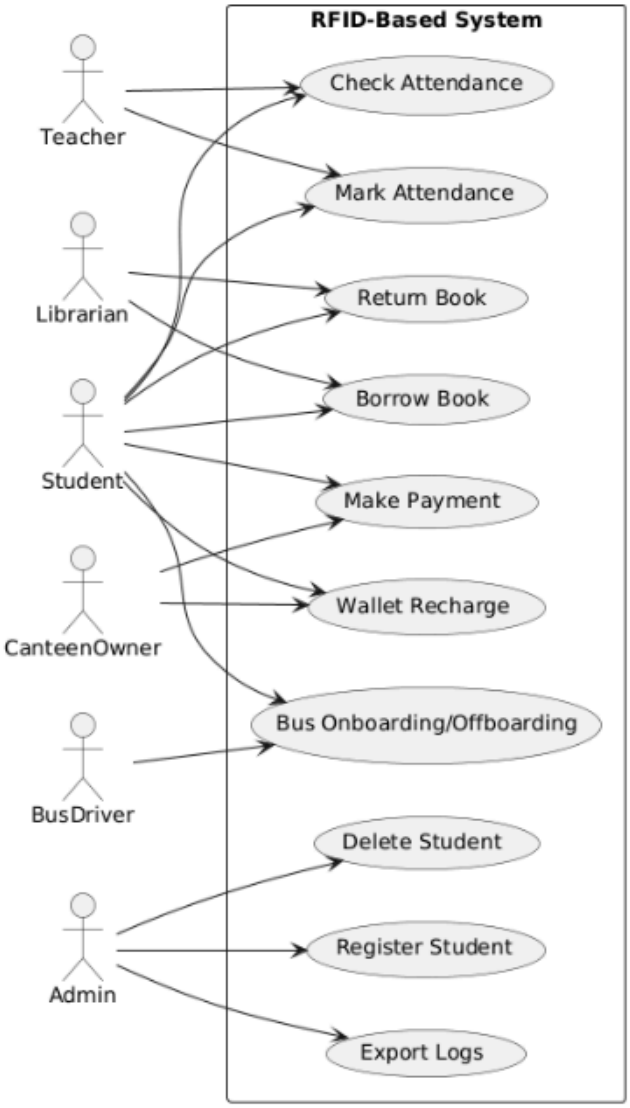


Figure 4.5: Use case diagram

- The student will have an RFID tag unique to him/her; this RFID tag will function as an ID for each student [5].
- Access is controlled based on **predefined roles** as shown in Figure 4.5:

Role	Permissions
Student	View logs (attendance, transactions, books, bus).
Teacher	Mark attendance, check attendance records.
Canteen	Process wallet transactions.
Librarian	Manage book borrowing/returns.
Driver	Log student onboarding/offboarding.
Admin	Manage users, reset logs, export reports.

4.6. STUDENT-BASED PROCESS WORKFLOW

Example - Attendance Process:

1. Student scans RFID card.
2. System retrieves student details.
3. Attendance is logged in the database.
4. Student receives confirmation.

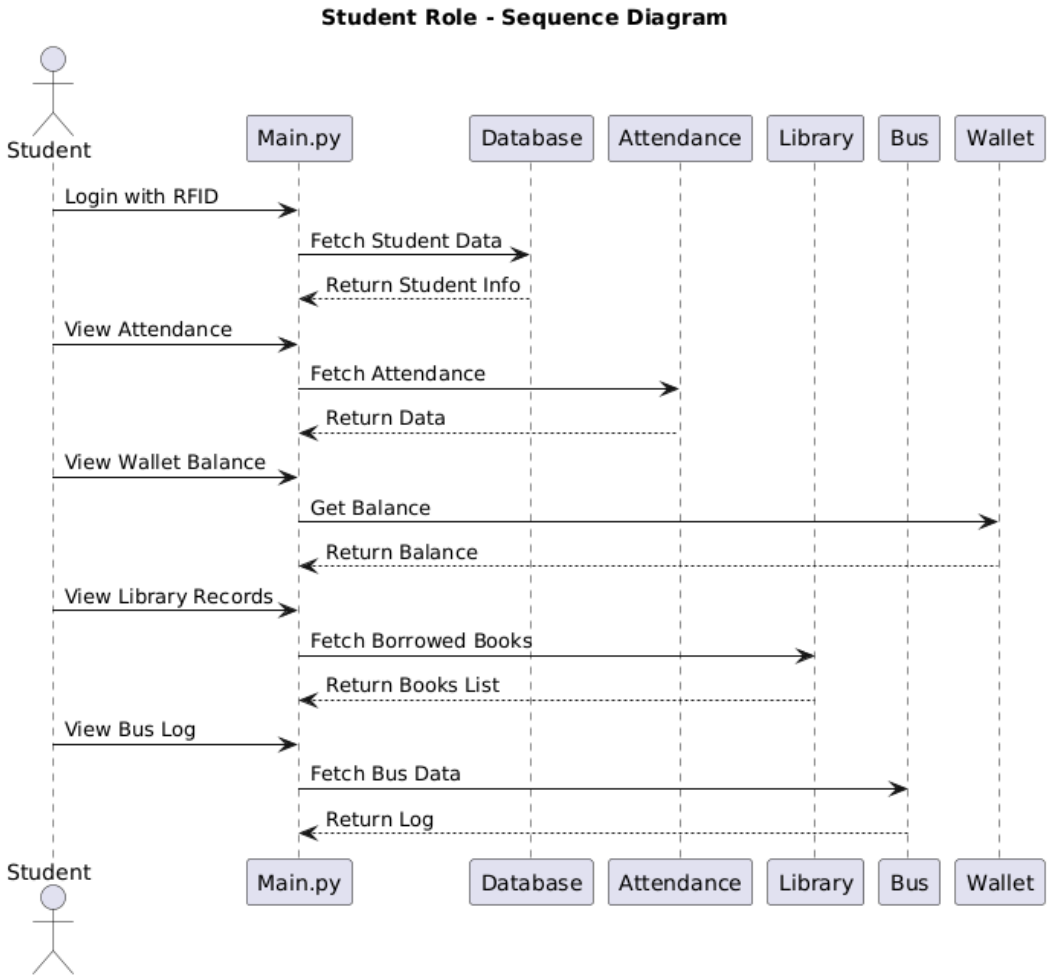


Figure 4.6: Sequence diagram for a student user

The **Student Sequence Diagram**, shown in Figure 4.6, illustrates a typical student interaction [10] (e.g., attendance marking, transaction processing).

5. IMPLEMENTATION

RADIT is architected as a modular Python desktop application, leveraging Tkinter for the graphical user interface and Firebase Firestore as the cloud backend. The codebase is organized into distinct modules, each encapsulated in its own Python class within the src/components/ directory. Shared logic, such as database access, RFID validation, and face recognition, is centralized in src/utils.py to promote code reuse and maintainability.

The application follows an event-driven design, where user actions in the GUI trigger backend logic, database operations, and hardware interactions. Each module (wallet, library, bus, attendance) operates independently but shares authentication and data resources, ensuring seamless integration and real-time data consistency.

5.1 DATABASE INTEGRATION

RADIT uses Firebase Firestore for all persistent data storage. The firebase-admin Python SDK is used to authenticate and interact with Firestore. Data is organized into collections such as students , transactions , books , lendings , bus_activity , and attendance . Each collection is structured with unique document IDs and relevant fields (e.g., student profiles, transaction details, book metadata).

CRUD operations are abstracted into utility functions in utils.py , ensuring consistent and secure data access. For example, when a student makes a wallet transaction, the transaction is recorded in the transactions collection, and the student's wallet balance is updated atomically. Real-time updates are leveraged for logs and notifications, and Firestore security rules restrict access based on user roles.

5.2 RFID AND BIOMETRIC INTEGRATION

RFID readers are interfaced via serial or USB using Python libraries. When a card is scanned, the UID is validated and mapped to a student record in Firestore. For attendance, OpenCV is used to capture and compare facial data. Face encodings are generated and stored in the database during student registration. During attendance marking, the system captures a live image, computes its encoding, and compares it to the stored encoding for verification, providing two-factor authentication .

5.3 RECOMMENDATION SYSTEMS

Pattern-based recommendation logic is implemented in Python utility functions:

- **Wallet Recharge:** The system analyzes the last 30 days of spending for each student, calculates weekly averages, and suggests a recharge amount. This logic is triggered when a student checks their wallet balance or receives a low balance alert.
- **Library Recommendations:** The system queries the student's borrowing history and suggests books from similar categories or based on what peers with similar interests have

borrowed. This is implemented as a background process that updates recommendations periodically or on demand.

5.4 USER INTERFACE

The user interface is built with Tkinter/ttk, providing a multi-window, event-driven experience:

- **Login and Authentication:** The login screen supports RFID card scanning for students and PIN/password entry [1, 16] for admins. Error handling is implemented for invalid credentials and hardware issues.
- **Dashboard:** After authentication, users are presented with a dashboard summarizing wallet balance, recent transactions, borrowed books, bus activity, and attendance status. Visual indicators (e.g., color-coded alerts, icons) highlight important information such as low balance or overdue books.
- **Module Windows:** Each module (wallet, library, bus, attendance) has its own window or tab, with forms for data entry, tables for data display, and dialogs for notifications. For example, the wallet module allows students to view transaction history, initiate recharges, and receive personalized suggestions.
- **Role-Based Access:** The UI dynamically adapts to the user's role, showing only relevant modules and options. Admins have access to management features such as student registration, book catalog management, and bus route configuration.
- **Feedback and Accessibility:** All actions provide immediate feedback via pop-ups or status bars. The UI uses large, readable fonts, clear icons, and consistent color schemes to enhance usability and accessibility.

5.5 SECURITY

Security is enforced at multiple levels:

- **RFID** for user identification and transaction authentication.
- **Facial recognition** for attendance, using OpenCV and stored face encodings.
- **PIN/password** for admin access and sensitive operations. **Sensitive operations** require re-authentication.

All data transmission to Firebase is secured via **HTTPS**, and **Firestore security rules** are enforced to prevent unauthorized access.

5.6 ERROR HANDLING AND RELIABILITY

The application includes robust error handling for hardware failures (e.g., RFID reader not detected), network issues (e.g., Firebase connectivity), and invalid user actions (e.g., duplicate book lending, insufficient wallet balance). User-friendly error messages are displayed, and logs are maintained for troubleshooting. The modular design allows for independent development, debugging, and extension of each component.

6. TESTING AND RESULTS

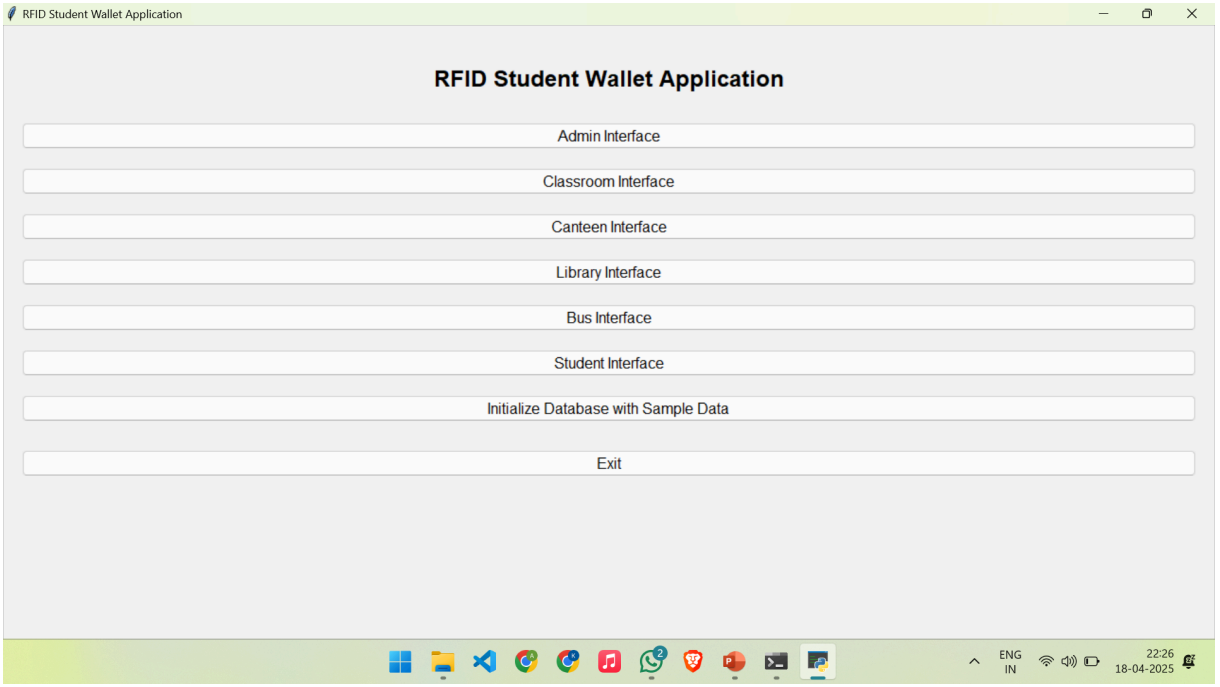


Figure 6.1: The main menu of the application

As shown in Figure 6.1, the main interface displays all RADIT modules in a centralized dashboard, providing access to digital wallet, library, bus tracking, and attendance systems.

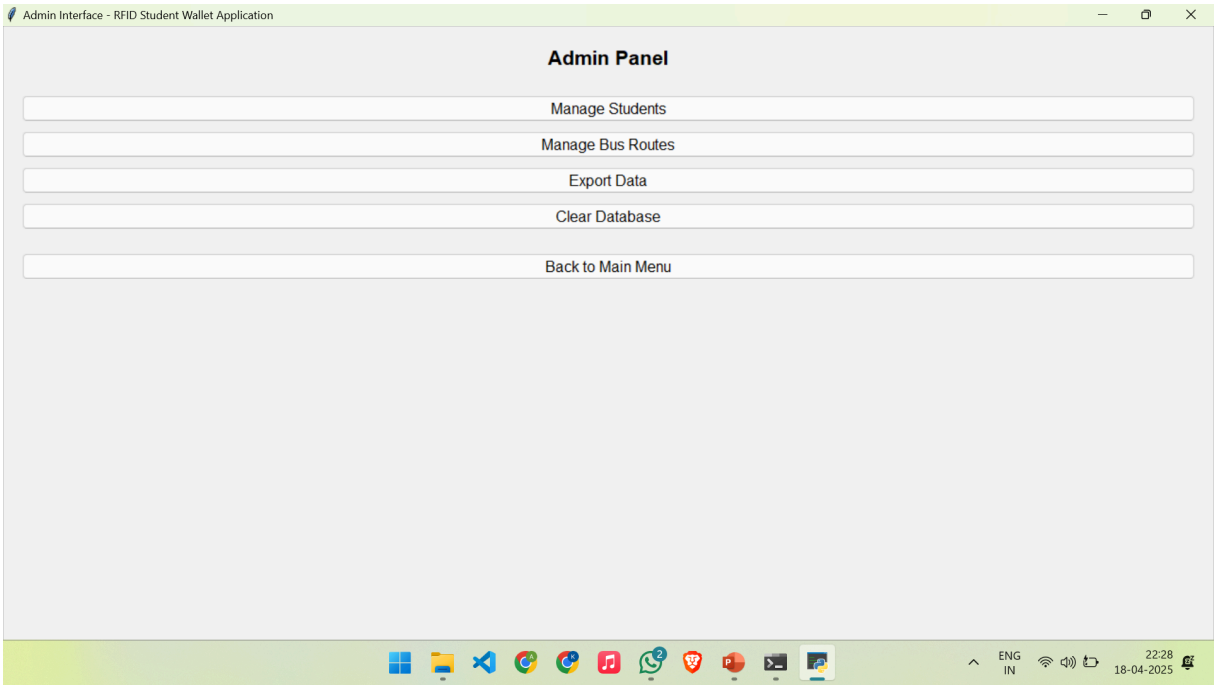


Figure 6.2: The admin interface

Figure 6.2 shows the administrator control panel for managing user accounts, system operations, and generating reports across all RADIT modules.

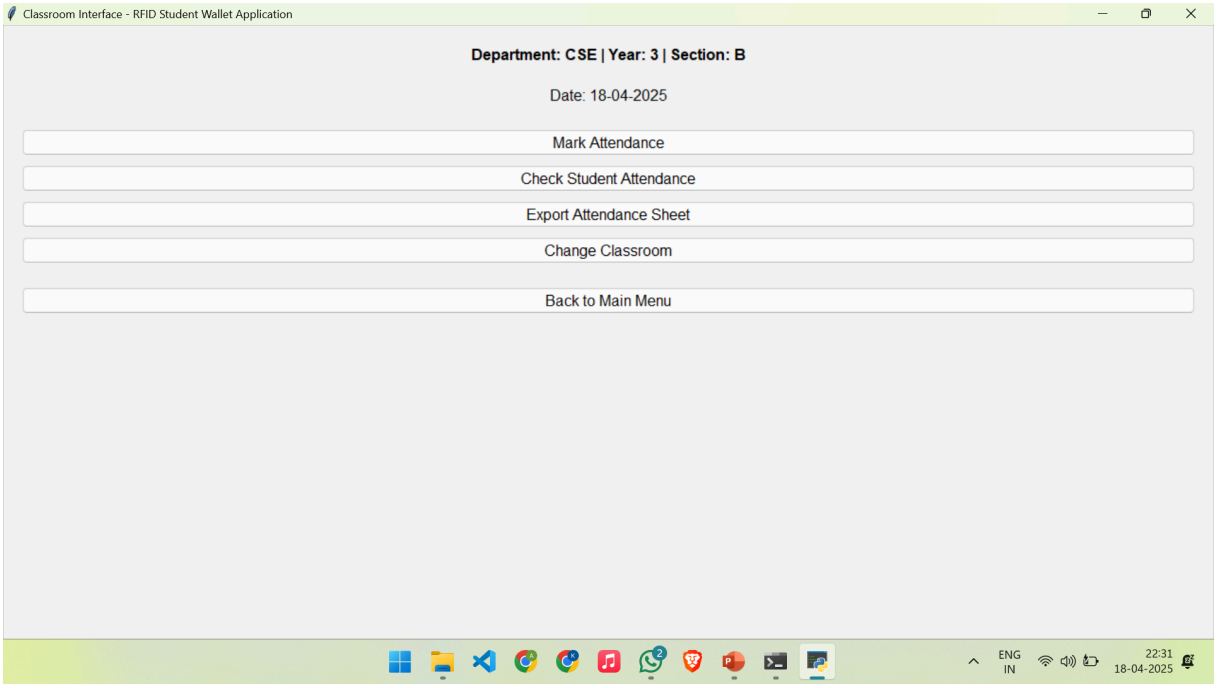


Figure 6.3: The classroom interface

As shown in Figure 6.3, the classroom interface allows instructors to view and mark student attendance using RFID or facial recognition technologies and also check individual student attendance.

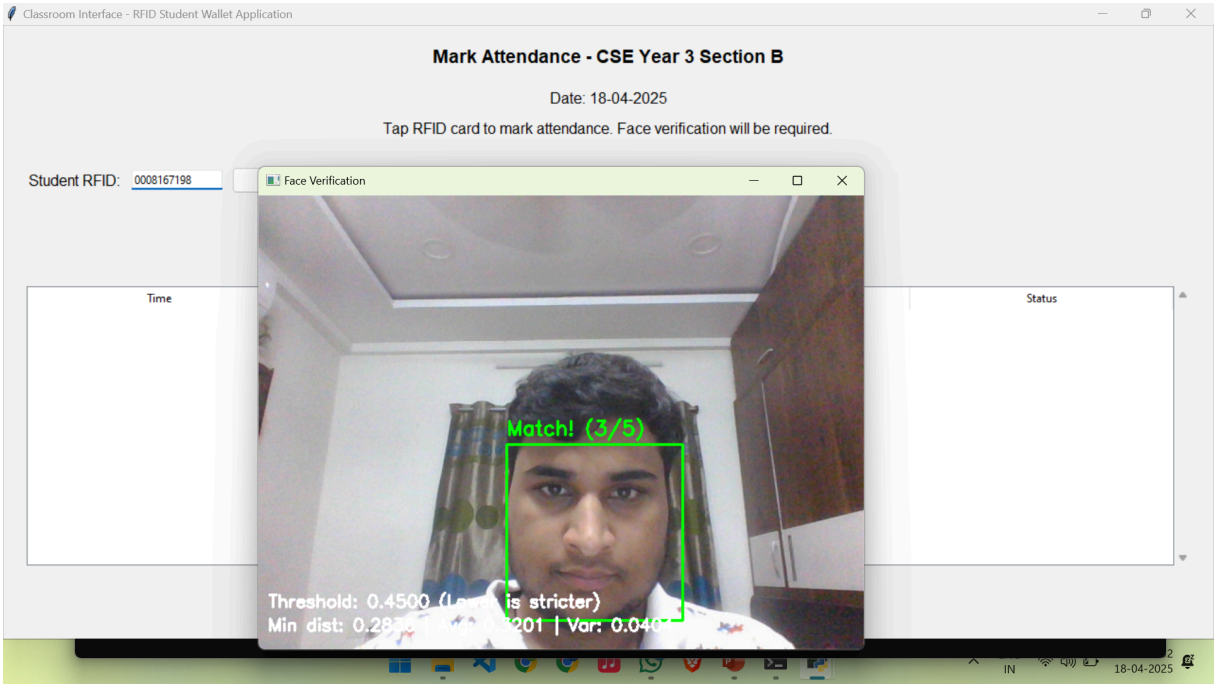


Figure 6.4: The hybrid attendance system

Figure 6.4 demonstrates the dual-verification attendance system that combines RFID scanning with facial recognition to prevent proxy attendance.

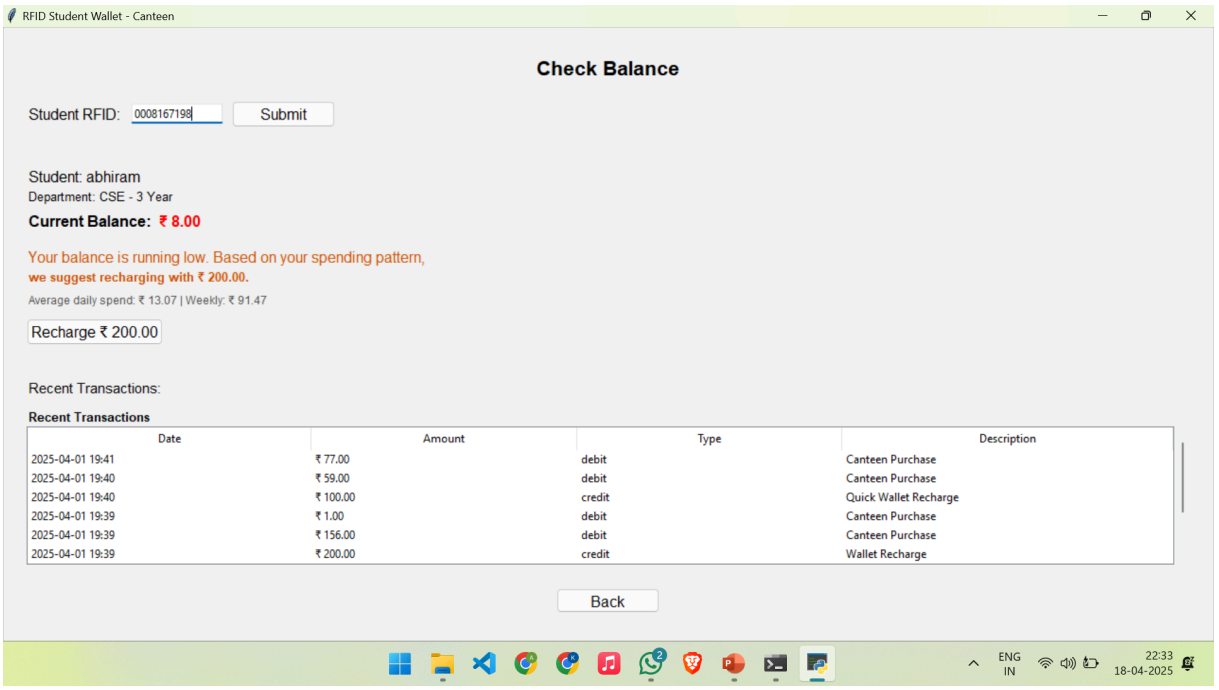


Figure 6.5: The canteen interface

As shown in Figure 6.5, the canteen interface displays wallet balance and provides intelligent recharge recommendations based on student spending patterns.

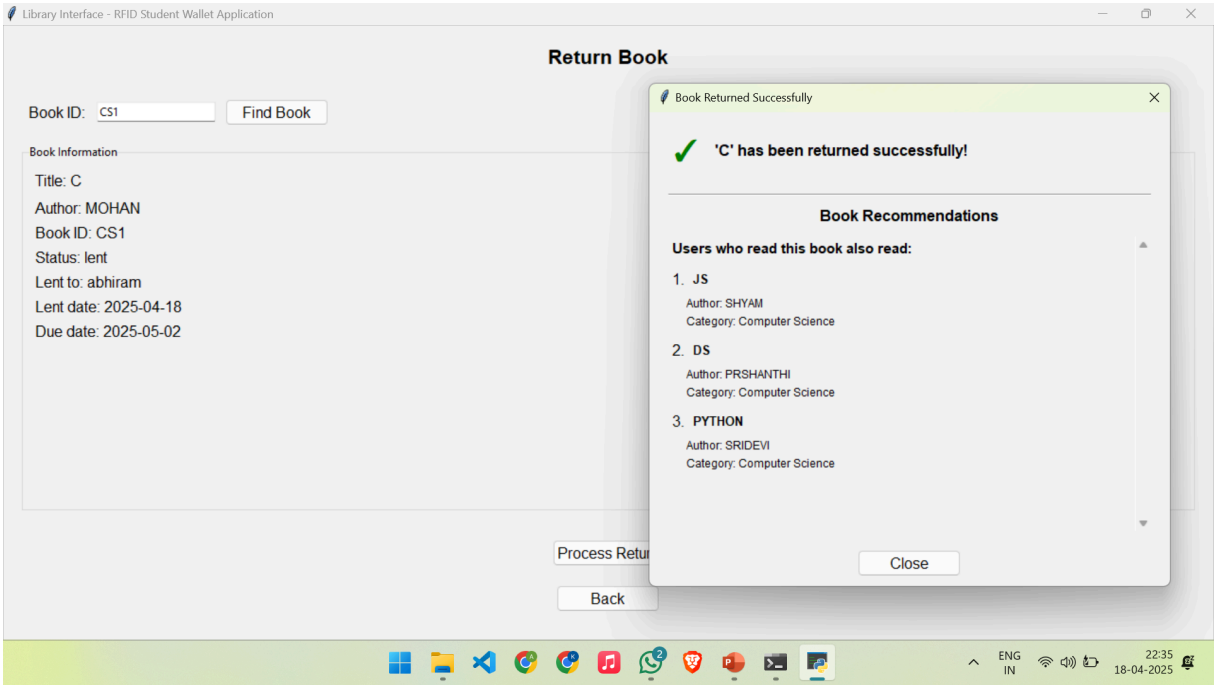


Figure 6.6: The library interface

Figure 6.6 shows the library management screen with book lending functionality and personalized reading recommendations based on student interests.

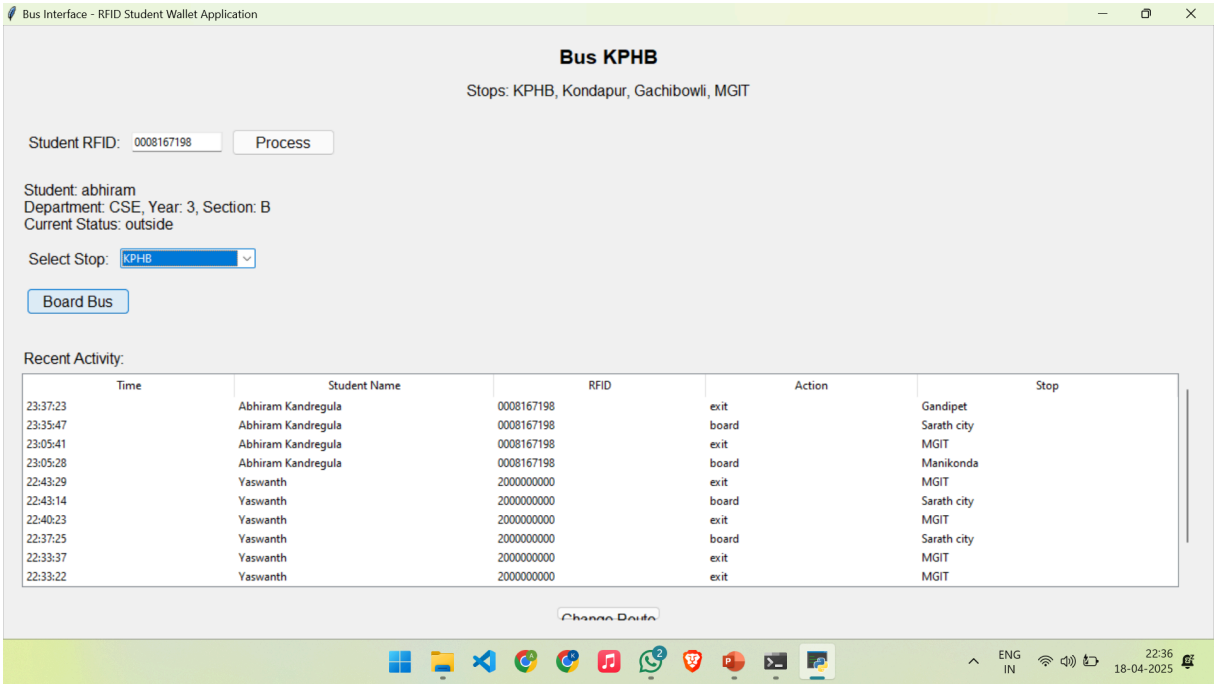


Figure 6.7: The bus interface

As shown in Figure 6.7, the transportation module tracks student bus boarding and exit records specific to the route number chosen.

7. CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

RADIT successfully demonstrates a unified, modular approach to campus management by integrating essential services—digital wallet, library management, bus tracking, and attendance—into a single, user-friendly platform. The use of Python, Tkinter, and Firebase Firestore ensures a robust, scalable, and real-time system. By leveraging RFID and biometric verification, RADIT enhances security and operational efficiency, while usage pattern-based recommendation systems improve the user experience for students and administrators alike. The modular architecture and cloud-based backend enable seamless data synchronization, centralized control, and easy extensibility. Overall, RADIT achieves its goal of reducing administrative chaos and creating a more organized, responsive, and student-centered campus environment.

7.2 FUTURE SCOPE

1. **Mobile Application:** Develop native or cross-platform mobile apps for students, parents, and staff to access RADIT services on the go.
2. **Web-Based Interface:** Extend the system with a modern web frontend for broader accessibility and easier administration.
3. **Expanded Biometric Options:** Incorporate additional biometric methods (e.g., fingerprint [4, 20], iris scan) for enhanced security.
4. **Offline Functionality:** Implement offline data caching and synchronization to ensure uninterrupted service during network outages.
5. **Third-Party Integrations:** Enable integration with other educational platforms, payment gateways, and transportation systems.
6. **Accessibility Enhancements:** Improve the user interface for better accessibility, including support for users with disabilities.
7. **Automated Notifications:** Expand notification channels (SMS, push notifications) for real-time alerts to students and parents.
8. **Customizable Modules:** Allow institutions to enable or disable modules based on their specific needs, making RADIT adaptable to various educational environments.

BIBLIOGRAPHY

[1] Onwubiko, Emmauel & S.E, Chaku & Kulugh, Victor & G.I.O, Aimufua, "Development of a Lecture Attendance Monitoring System with Multi-Level Authentication", *Science World Journal*, vol. 20, 2025, pp. 230-236. (https://doi.org/10.4314/swj.v20i1.30)

[2] Ranjan, Rashmi & Josephine, Christina & Moses, M & Sona, Deepika & M, Aarthy. (2024). "IoT-Based School Bus and Student Monitoring System Using RFID and GSRM Technologies", *International Journal of Intelligent Systems and Applications in Engineering*. 12. 164-173. (https://www.ijisae.org/index.php/IJISAE/article/view/5237)

[3] Lucky Oji Akpojotor & Esoswo Francisca Ogbomo, "Enhancing Library Management through RFID and IoT Integration in Nigeria: Benefits, Challenges, and Future Prospects", *Journal of African Information Systems and Technology Online*, vol. 18, no. 1, 2024, pp. 1-15. (https://doi.org/10.70118/jaist.202501801.4)

[4] N. R et al., "Biometric and RFID Passive Tag based Student Identification System for Secure Attendance Management", 2023, 4th *International Conference on Intelligent Engineering and Management (ICIEM)*, London, United Kingdom, 2023, pp. 1-6. (https://doi.org/10.1109/ICIEM59379.2023.10166924)

[5] Farag, W. A., "An RFID-based smart school attendance and monitoring system", 2023, *BOHR Journal of Computational Intelligence and Communication Network*, 1(1), 26–34. (https://doi.org/10.54646/bjcicn.2023.05)

[6] Srinivasan, Rajasekaran & Mohan, M & Muthusamy, Ganesamoorthy & Palanisamy, Selvakamal & Librarian, Assistant & Librarian (2023). "RFID-Based Library Management System used in Library." *ResearchGate* 8. 26-32. (https://www.researchgate.net/publication/369541235_RFID-Based_Library_Management_System_used_in_Library)

[7] Samaddar, Rajarshi & Ghosh, Aikyam & Sarkar, Sounak & Das, Mainak & Chakrabarty, Avijit, "IoT & Cloud-based Smart Attendance Management System using RFID", *International Research Journal on Advanced Science Hub*, vol. 5, 2023, pp. 111-118. (http://dx.doi.org/10.47392/irjash.2023.020)

[8] Yogayya Swami, Mahesh Dawlar & Prof. Nita Dimble, "AI System for Avoid Proxy Students", *International Journal of Professional Research and Practice*, ISSN 2582-7421, 2023. (https://ijrpr.com/uploads/V6ISSUE3/IJRPR40573.pdf)

[9] Komal Sukraj Dambre, Yogita Sanjay Gend, Prachi Atmaram Kadam, Harshita Madhukar Shewale & Prof. B.M. Gawale, "Smart Bus Tracking System for Students using RFID", *International Journal of Professional Research in Engineering, Management and Science*, 2023. (https://www.ijprems.com/uploadedfiles/paper//issue_5_may_2024/33998/final/fin_ijprems1715496959.pdf)

[10] Ike, Ifeanyi, "Design and Implementation of Student Information System", *ResearchGate*, 2023. (http://dx.doi.org/10.13140/RG.2.2.16736.87044)

[11] Helaimia, Rafika, "Cloud Computing In Higher Education Institutions: Pros and Cons", *ResearchGate,* 2023, pp. 132-141.

(https://www.researchgate.net/publication/374753105_Cloud_Computing_In_Higher_Education_Institutions_Pros_and_Cons)

[12] Aditi Akundi, Aditi Kulkarni, Ajay H.R, Akash R, Pavithra G & T.C.Manjunath, "Cloud-based Attendance Management System with Integrated RFID Data Acquisition and Real-Time Google Sheets Synchronization", *Grenze International Journal of Engineering and Technology*, June Issue, 2023. (https://thegrenze.com/pages/servej.php?fn=498.pdf&name=Cloud-based%20Attendance%20Management%20System%20withIntegrated%20RFID%20Data%20Acquisition%20and%20Real-TimeGoogle%20Sheets%20Synchronization&id=3575&association=GRENZE&journal=GIJET&year=2024&volume=10&issue=2)

[13] P. Naresh, Samavedam Venkataramana Naga Pavan, Abdul Razzakh Mohammed, Modepu Tharun & Nenavath Chanti, "Implementing a Flask-based Chatbot for College Enquiries using Spacy and TensorFlow", *Journal of Engineering Sciences ICETT*, vol. 14, no. 05(S), 2023. (https://jespublication.com/specialissue/2023-V14I5010.pdf)

[14] Valentina Terzieva, Svetozar Ilchev, Katia Todorova, "The Role of Internet of Things in Smart Education", *IFAC*, Volume 55, Issue 11, 2022, Pages 108-113, ISSN 2405-8963. (https://doi.org/10.1016/j.ifacol.2022.08.057)

[15] Chelvarayan A, Yeo SF, Hui Yi H, Hashim H., "E-Wallet: A Study on Cashless Transactions Among University Students", *F1000Res*, 2022 Jun 21;11:687. (https://f1000research.com/articles/11-687/v1)

[16] Kumar, V.; Kumar, R.; Khan, A.A.; Kumar, V.; Chen, Y.-C.; Chang, C.-C. "RAFI: Robust Authentication Framework for IoT-Based RFID Infrastructure.", *MDPI,* Sensors 2022, 22, 3110. (https://doi.org/10.3390/s22093110)

[17] Shen, Xiaoying & Yuan, Chao. (2021). "A College Student Behavior Analysis and Management Method Based on Machine Learning Technology", *Wireless Communications and Mobile Computing*. 2021. (https://doi.org/10.1155/2021/3126347)

[18] Aslam, Fankar & Mohammed, Hawa & Lokhande, Prashant, "Efficient Way Of Web Development Using Python And Flask", *International Journal of Advanced Research in Computer Science*, vol. 6, 2015. (https://ijarcs.info/index.php/Ijarcs/article/view/2434)

[19] Kashish Ara Shakil, Shuchi Sethi & Mansaf Alam, "An Effective Framework for Managing University Data using a Cloud based Environment", *arXiv* preprint arXiv:1501.07056, 2015. (https://doi.org/10.48550/arXiv.1501.07056)

[20] Bhattacharyya, Debnath & Rahul, Ranjan & Alisherov, Farkhod & Minkyu, Choi, "Biometric Authentication: A Review", *International Journal of u- and e- Service*, Science and Technology, vol. 2, 2009. (https://www.researchgate.net/publication/46189709_Biometric_Authentication_A_Review)

2.

APPENDIX A

1. Main Application Initialization (main.py)

```
# Initialize Firebase
try:
    # Check if firebase config file exists
    if not os.path.exists('firebase_config.json'):
        # Create placeholder config
        firebase_config = { ... }
        with open('firebase_config.json', 'w') as f:
            json.dump(firebase_config, f, indent=4)

    # Initialize Firebase with service account
    cred = credentials.Certificate("serviceAccountKey.json")
    firebase_admin.initialize_app(cred)
    db = firestore.client()
except Exception as e:
    print(f"Error initializing Firebase: {e}")
    messagebox.showerror("Firebase Error", f"Could not initialize Firebase: {e}")
    db = None

def initialize_database():
    """Initialize database with sample data."""
    # Sample data structures
    admin_data = { ... }
    students_data = [ ... ]
    routes_data = [ ... ]
    books_data = [ ... ]

    # Helper function to add data
    def add_collection_data(collection_name, data_list):
        collection_ref = db.collection(collection_name)
        for data in data_list:
            doc_ref = collection_ref.add(data)

    # Add sample data to collections
    add_collection_data("admin", [admin_data])
    add_collection_data("students", students_data)
    add_collection_data("routes", routes_data)
    add_collection_data("books", books_data)
```

2. Core Utilities (utils.py)

```
#RFID Validation and Authentication
def validate_rfid(rfid):
    """Validate RFID format (10 digits)"""
    return rfid and re.match(r'^\d{10}$', rfid)

def get_student_by_rfid(db, rfid):
    """Get student document from Firebase by RFID"""
    students_ref = db.collection('students')
    query = students_ref.where(filter=firestore.FieldFilter('rfid', '==',
rfid)).limit(1)
    results = query.get()
    for doc in results:
        return {'id': doc.id, **doc.to_dict()}
    return None
```

```
#Face Recognition System
def capture_face(camera_index=0, required_encodings=7):
    """Capture and encode a face using camera"""
    cap = cv2.VideoCapture(camera_index)
    face_encodings = []
    while len(face_encodings) < required_encodings:
        ret, frame = cap.read()
        rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        face_locations = face_recognition.face_locations(rgb_frame, model="hog")
        if len(face_locations) == 1:
            new_encodings = face_recognition.face_encodings(rgb_frame,
face_locations)
            if new_encodings:
                face_encodings.append(new_encodings[0])
    return face_encodings

def verify_face(known_face_encodings, camera_index=0, tolerance=0.45):
    """Verify face against stored encoding"""
    cap = cv2.VideoCapture(camera_index)
    verification_result = False
    while not verification_result:
        ret, frame = cap.read()
        rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        face_locations = face_recognition.face_locations(rgb_frame, model="hog")
        if len(face_locations) == 1:
            current_face_encoding = face_recognition.face_encodings(rgb_frame,
face_locations)[0]
            face_distance = face_recognition.face_distance(known_face_encodings,
current_face_encoding)
            if min(face_distance) <= tolerance:
                verification_result = True
    return verification_result

#UI Component Generation
def create_rfid_frame(parent, callback):
    """Create standard RFID input frame"""
    frame = ttk.Frame(parent)
    label = ttk.Label(frame, text="Tap or Enter RFID:")
    entry = ttk.Entry(frame, width=15)
    button = ttk.Button(frame, text="Submit",
                        command=lambda: callback(entry.get()))
    # ... component layout ...
    return frame, entry

def create_entry_with_label(parent, label_text, row, column=0, width=20,
show=None):
    """Create labeled entry field"""
    label = ttk.Label(parent, text=label_text)
    entry = ttk.Entry(parent, width=width)
    # ... grid layout ...
    return entry
```

3. User Interfaces:

i. Bus UI (bus_ui.py)

```
 def process_boarding(self, student):
     """Process student boarding the bus"""
     selected_stop = self.stop_var.get()
```

```
        if not selected_stop:
            messagebox.showerror("Missing Information", "Please select a stop.")
            return

        # Check if student is already marked as boarded
        if student.get('bus_status') == 'inside':
            confirm = messagebox.askyesno("Already Boarded",
                                          f"Update boarding information?")
            if not confirm: return

        try:
            now = datetime.datetime.now()
            # Update student status in Firestore
            self.db.collection('students').document(student['id']).update({
                'bus_status': 'inside',
                'last_bus_action': {
                    'action': 'board',
                    'timestamp': now,
                    'route': self.route_data['route_id'],
                    'stop': selected_stop
                }
            })

            # Record bus activity
            activity_data = {
                'student_id': student['id'],
                'student_name': student.get('name'),
                'action': 'board',
                'route_num': self.route_data['route_id'],
                'stop': selected_stop,
                'timestamp': now
            }
            self.db.collection('bus_activity').add(activity_data)

            # Send email notification to parent
            if parent_email := student.get('parent_email'):
                message = f"{student.get('name')} boarded bus at {selected_stop}"
                send_email(parent_email, "Bus Boarding Notification", message)

            messagebox.showinfo("Success", "Boarding processed successfully")
            self.load_recent_activity()  # Refresh activity log

        except Exception as e:
            messagebox.showerror("Error", f"Boarding failed: {e}")
```

ii. Student UI (student_ui.py)

```
  def display_student_info(self, student):
        """Display comprehensive student information"""
        # Create scrollable canvas for information display
        canvas = tk.Canvas(self.main_frame)
        scrollbar = ttk.Scrollbar(self.main_frame, command=canvas.yview)
        canvas.configure(yscrollcommand=scrollbar.set)

        # Display personal details
        ttk.Label(info_frame, text=f"Name:
    {student.get('name')}").pack(anchor=tk.W)
```

```
        ttk.Label(info_frame, text=f"Department:
    {student.get('department')}").pack(anchor=tk.W)

        # Show wallet balance with color coding
        balance = student.get('wallet_balance', 0)
        color = "green" if balance > 200 else "orange" if balance > 50 else "red"
        ttk.Label(info_frame, text=f"Balance: {format_currency(balance)}",
                  foreground=color, font=("Helvetica", 12,
    "bold")).pack(anchor=tk.W)

        # Display attendance percentage
        self.display_attendance_info(info_frame, student['id'])

        # Show borrowed books
        self.display_library_info(info_frame, student['id'])

        # Display recent activity in a Treeview
        columns = ("date", "type", "details")
        activity_tree = ttk.Treeview(activity_frame, columns=columns,
    show="headings")
        activity_tree.heading("date", text="Date")
        activity_tree.heading("type", text="Type")
        activity_tree.heading("details", text="Details")
        self.load_recent_activity(activity_tree, student['id'])
```

iii. Classroom UI (classroom_ui.py)

```
    def process_attendance(self, student):
        """Record student attendance with face verification"""
        # Capture student's face
        face_image = capture_face()

        # Verify against registered face data
        if not verify_face(student['face_data'], face_image):
            messagebox.showerror("Verification Failed", "Face doesn't match!")
            return

        # Record attendance in Firestore
        attendance_data = {
            'student_id': student['id'],
            'classroom_id': self.classroom_info['id'],
            'status': 'present',
            'timestamp': datetime.datetime.now()
        }
        self.db.collection('attendance').add(attendance_data)

        messagebox.showinfo("Success", "Attendance recorded!")
        self.student_rfid_entry.delete(0, tk.END)  # Clear for next student
```

iv. Library UI (library_ui.py)

```
    def process_book_return(self):
        """Handle book return with fine calculation"""
        book_id = self.return_book_id_entry.get().strip()
        if not book_id: return

        try:
            # Find lending record
            lending_ref = self.db.collection('lendings').where(
```

```
            filter=firestore.FieldFilter('book_id', '==', book_id)
        ).where(
            filter=firestore.FieldFilter('status', '==', 'active')
        ).limit(1).get()[0]

        lending_data = lending_ref.to_dict()
        due_date = lending_data['due_date']
        days_late = (datetime.datetime.now() - due_date).days

        # Calculate fine if overdue
        fine = max(0, days_late * FINE_PER_DAY) if days_late > 0 else 0

        # Update lending record
        lending_ref.reference.update({
            'return_date': datetime.datetime.now(),
            'fine': fine,
            'status': 'returned'
        })

        # Update book status
        self.db.collection('books').document(book_id).update({
            'status': 'available',
            'borrower': firestore.DELETE_FIELD
        })

        # Show return confirmation
        messagebox.showinfo("Success", f"Book returned. {'Fine: ₹'+str(fine)
if fine else 'No fine'}")

    except Exception as e:
        messagebox.showerror("Error", f"Return failed: {e}")
```

v. Canteen UI (canteen_ui.py)

```
def process_payment(self, student):
    """Process canteen purchase transaction"""
    try:
        amount = float(self.amount_entry.get())
        if amount <= 0: return

        # Check sufficient balance
        if amount > student.get('wallet_balance', 0):
            messagebox.showerror("Error", "Insufficient balance!")
            return

        # Create transaction
        transaction = {
            'student_id': student['id'],
            'amount': amount,
            'type': 'debit',
            'description': self.desc_entry.get() or 'Canteen Purchase',
            'timestamp': datetime.datetime.now()
        }

        # Update student balance
        new_balance = student['wallet_balance'] - amount
        transaction_batch = self.db.batch()
        transaction_ref = self.db.collection('transactions').document()
        transaction_batch.set(transaction_ref, transaction)
        transaction_batch.update(
```

```
                self.db.collection('students').document(student['id']),
                {'wallet_balance': new_balance}
            )
            transaction_batch.commit()

            messagebox.showinfo("Success", f"Payment processed. New balance:
₹{new_balance:.2f}")
            self.refresh_transaction_display(student['id'])

        except ValueError:
            messagebox.showerror("Error", "Invalid amount entered")
```

vi. Admin UI (admin_ui.py)

```
    def add_student(self):
        """Admin interface for adding new students"""
        # Collect student data from form
        student_data = {
            'name': self.name_entry.get(),
            'rfid': self.rfid_entry.get(),
            'department': self.dept_entry.get(),
            'year': int(self.year_entry.get()),
            'face_data': self.capture_face_data()  # Special face enrollment
        }

        # Validate RFID format
        if not validate_rfid(student_data['rfid']):
            messagebox.showerror("Error", "Invalid RFID format")
            return

        # Add to Firestore
        try:
            self.db.collection('students').add(student_data)
            messagebox.showinfo("Success", "Student added successfully")
            self.manage_students()  # Return to management view
        except Exception as e:
            messagebox.showerror("Database Error", str(e))
```

4. Logic for the Top-Up Suggestion System:

```
def get_spending_pattern(db, student_id, days=30):
    from datetime import datetime, timedelta
    try:
        start_date = datetime.now() - timedelta(days=days)
        txs = db.collection('transactions').where(
            filter=firestore.FieldFilter('student_id', '==', student_id)
        ).get()

        spends = [tx.to_dict() for tx in txs if tx.to_dict().get('type') ==
'debit' and
                  tx.to_dict().get('timestamp',
datetime.min).replace(tzinfo=None) >= start_date]
        if not spends: return None

        total = sum(tx.get('amount', 0) for tx in spends)
        daily = total / days
        return {'daily_avg': daily, 'weekly_avg': daily * 7}
    except:
        return None
```

```
def recommend_recharge_amount(pattern):
    return round((pattern['weekly_avg'] * 2 if pattern else 500) / 100) * 100
```

5. Logic for Library Book Recommendation System:

```
# Simplified Logic for finding similar books
def get_similar_books_logic(db, book_id):
    # Get the current book's category
    current_book_category = db.get_book_category(book_id)

    # Find other books in the same category that are available
    category_recommendations =
db.find_available_books_by_category(current_book_category)

    # Find students who borrowed the current book
    students_who_read_this = db.get_students_who_borrowed(book_id)

    # Find other books borrowed by those students that are available
    user_history_recommendations =
db.find_available_books_borrowed_by_students(students_who_read_this)

    # Combine and prioritize recommendations (e.g., history-based first, then
category)
    combined_recommendations = prioritize(user_history_recommendations,
category_recommendations)

    # Return a limited list of recommendations
    return combined_recommendations[:max_recommendations]

# Simplified Logic for personalized book recommendations
def get_user_recommendations_logic(db, student_id):
    # Get the student's reading history (books they've returned)
    student_history = db.get_returned_books_by_student(student_id)

    # Identify the categories the student reads most often
    preferred_categories = analyze_reading_categories(student_history)

    # Find available books from the student's preferred categories that they
haven't read
    recommendations = db.find_available_books_by_categories(preferred_categories)

    # If not enough, add some random available books
    if len(recommendations) < max_recommendations:
        recommendations.extend(db.get_random_available_books())

    # Return a limited list of unique recommendations
    return unique_and_limit(recommendations, max_recommendations)
```

APPENDIX B

User manual to run the application. Initially, clone the repository from https://github.com/AbhiramK01/RFID_Student_Wallet on GitHub onto your local system.

Installation:

1. Prerequisites:
   - Python 3.8 or higher
   - Webcam for facial recognition
   - Firebase account
   - Git (download and install from https://git-scm.com/)
2. Setup Steps (open command prompt or terminal on your system and run the command from the instructions given below):
   a. Clone the repository

```
i.   git init

ii.  git clone
     https://github.com/AbhiramK01/RFID_Student_Wallet.gi
     t

iii. cd RFID_Student_Wallet
```

   b. Create and activate virtual environment (recommended)

```
i.   python -m venv venv

ii.  source venv/bin/activate  # On Windows:
     venv\Scripts\activate
```

   c. Install dependencies

```
i.   pip install -r requirements.txt
```

Firebase Setup:

This application uses Firebase for backend services:

1. Create a Firebase Project:
   - Go to https://console.firebase.google.com/
   - Create a new project
2. Set Up Firestore Database:
   - Enable Firestore Database in test mode
3. Generate Service Account Credentials:
   - Go to Project Settings > Service Accounts
   - Generate and download the private key JSON file
   - Save it as serviceAccountKey.json in the project root directory (/rfid-student-wallet/serviceAccountKey.json)

4. Configure the Application:

    - Create a firebase_config.json file also in the root directory (/rfid-student-wallet /firebase_config.json) with your Firebase project details which are available in Project Overview>Project Settings>General>Your apps>SDK setup and configuration>Config

      Note: Omit the "`const firebaseConfig =`" part from the config code given and only copy and paste the content between the curly braces including the the curly braces ({}) as shown below.

      Format example:

```
{
  apiKey: "AIzaSyDj0LWvQCe9J8hena2KY2lrf9vqEL25hN8",
  authDomain: "fir-62f85.firebaseapp.com",
  databaseURL:
"https://fir-62f85-default-rtdb.firebaseio.com",
  projectId: "fir-62f85",
  storageBucket: "fir-62f85.firebasestorage.app",
  messagingSenderId: "982097276858",
  appId: "1:982097276858:web:47d976c3194c91b337b783",
  measurementId: "G-16N7XLY86L"

            };
```

Running the Application:
Note: Make sure your virtual environment is activated and run the below command in the same directory in which you ran the command to install the dependencies.
`> python run.py`
Interfaces:

- Admin: Manage students and system settings
- Classroom: Take attendance
- Canteen: Process wallet transactions
- Library: Handle book lending/returns
- Bus: Track transportation
- Student: Access student services

Note: Admin RFID for accessing and creating accounts is *0006435835*